

Công nghệ Tiếng nói

Course ID: AI409DV01

Lecturer: Bùi Ngọc Lê

Email: le.buingoc@hoasen.edu.vn

Chủ đề

Deep Learning

Nội dung

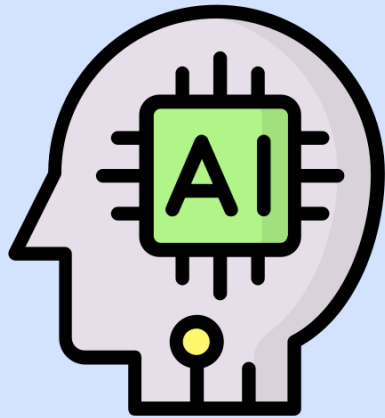
- Giới thiệu Deep Learning
- Cơ bản về mạng nơ-ron
- Perceptron

Giới thiệu Deep Learning

- Là phương pháp học máy sử dụng mạng nơ-ron nhân tạo để trích xuất đặc trưng tự động từ dữ liệu

Trí tuệ nhân tạo

Bất kỳ kỹ thuật nào cho phép máy tính mô phỏng hành vi của con người



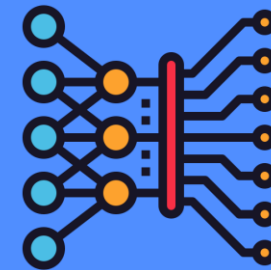
Máy học

Khả năng học tập mà không cần lập trình rõ ràng



Học sâu

Trích xuất đặc trưng bằng mạng nơ-ron.



Giới thiệu Deep Learning

- Năm 2015, Goodfellow đã tạo bước ngoặt lớn cho Deep Learning với đề xuất mô hình Generative Adversarial Networks (GAN).
- Năm 2018, Tero Karras, Samuli Laine và Timo Aila đã tạo ra cách mạng cho Generative Adversarial Networks với đề xuất phương pháp huấn luyện GAN bằng cách tăng dần độ phân giải của ảnh trong quá trình học
- Năm 2020, GPT-3 của OpenAI ra mắt với 175 tỷ tham số, đánh dấu bước nhảy vọt về khả năng sinh ngôn ngữ tự nhiên

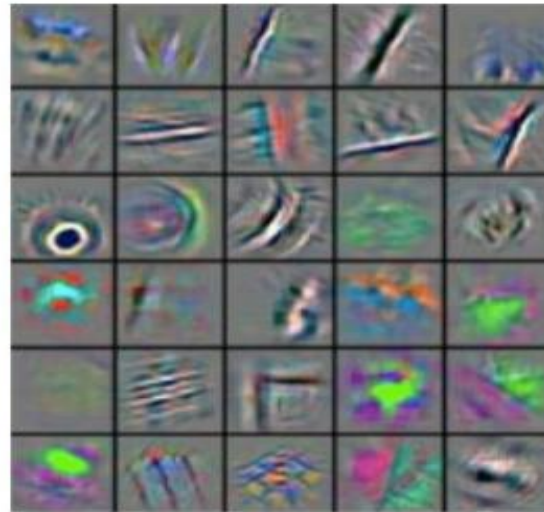
Giới thiệu Deep Learning

- Tại sao phải cần Deep Learning?
 - Phương pháp học máy truyền thống đòi hỏi trích xuất đặc trưng một cách thủ công, đòi hỏi kinh nghiệm và phụ thuộc từng bài toán cụ thể
 - Học sâu cho phép trích chọn đặc trưng tự động từ dữ liệu

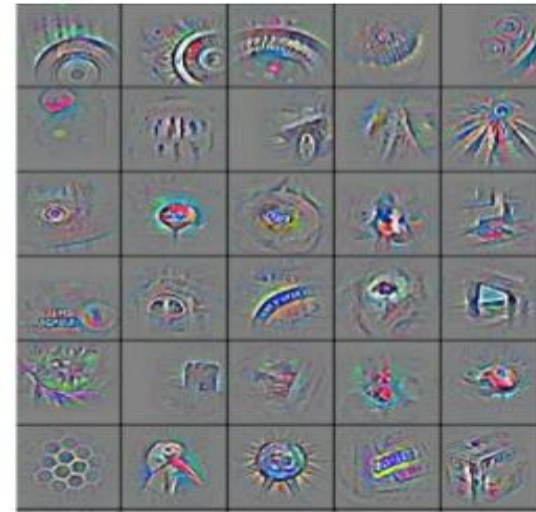
low-level features



mid-level features



high-level features



Giới thiệu Deep Learning

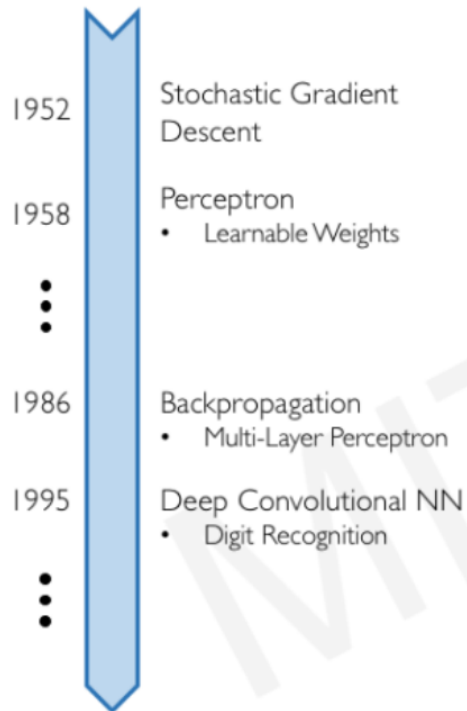
- Sự bùng nổ của Deep Learning

Mạng nơ-ron đã có từ nhiều thập kỷ trước

Big Data: thu thập và lưu trữ dễ dàng

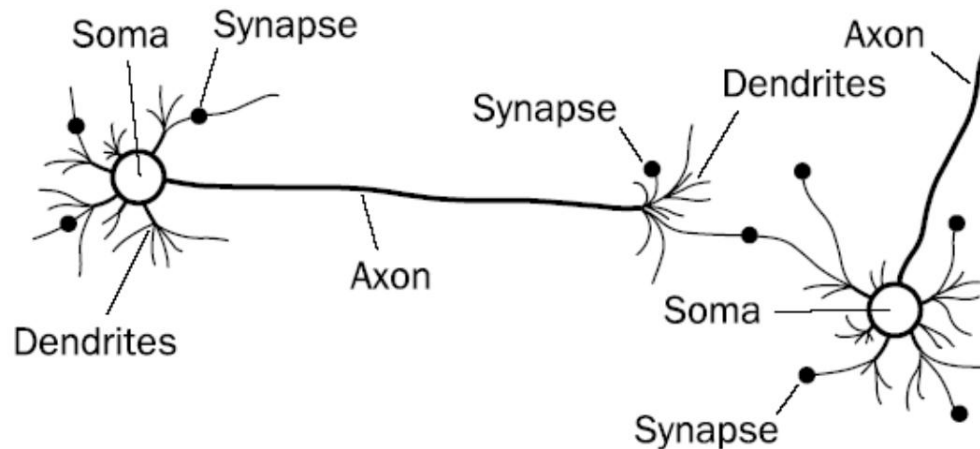
Công nghệ phần cứng phát triển

Các thư viện nền tảng phát triển



Cơ bản về mạng nơ-ron

- Não bộ con người gồm:
 - 10^{10} tế bào thần kinh – neuron, 6×10^{14} khớp thần kinh synapse.
 - Mỗi tế bào có cấu trúc đơn giản.
 - Một neuron bao gồm: thân – soma, nhiều sợi thần kinh dendrite và một sợi trục chính – axon.

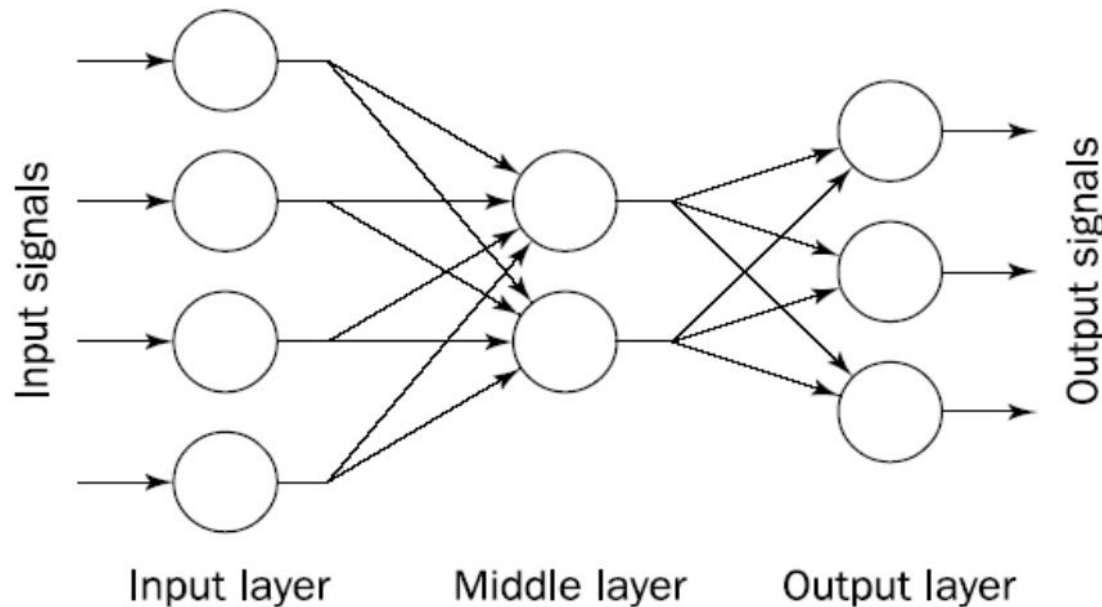


Cơ bản về mạng nơ-ron

- Cơ chế học của neuron thần kinh:
 - Tín hiệu được lan truyền giữa các neuron.
 - Một neuron nhận tín hiệu kích thích từ các khớp nối và phát tín hiệu qua soma đến các neuron khác.
 - Mỗi liên hệ giữa các neuron (bộ nhớ dài hạn) quy định chức năng của mạng neuron và được hình thành từ từ qua quá trình học.

Cơ bản về mạng nơ-ron

- Mạng neuron nhân tạo (Artificial Neural Network – ANN):
 - Là một tập hợp các bộ xử lý rất đơn giản – neuron – và nối với nhau.
 - Cấu trúc và phương thức hoạt động của ANN mô phỏng tương tự mạng neuron sinh học.



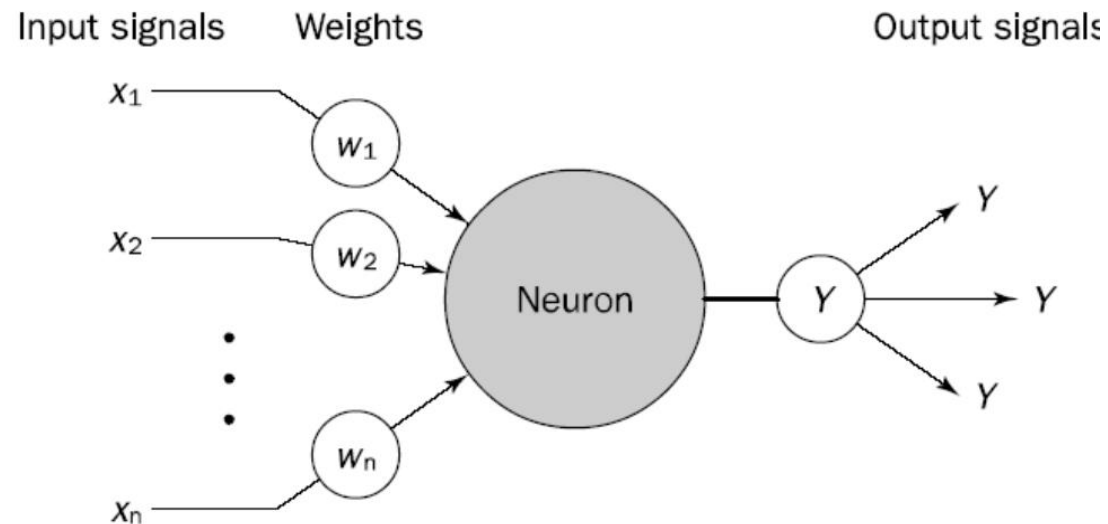
Cơ bản về mạng nơ-ron

- Các neuron được nối với nhau bằng các trọng số tương ứng. Các trọng số ứng với bộ nhớ dài hạn của ANN.
- ANN “học” bằng cách điều chỉnh từ từ các trọng số này qua quá trình tương tác với môi trường (huấn luyện).

Mạng neuron sinh học	Mạng neuron nhân tạo
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight (trọng số)

Cơ bản về mạng nơ-ron

- Neuron – Thành phần tính toán đơn giản



$$X = \sum_{i=1}^n x_i w_i$$

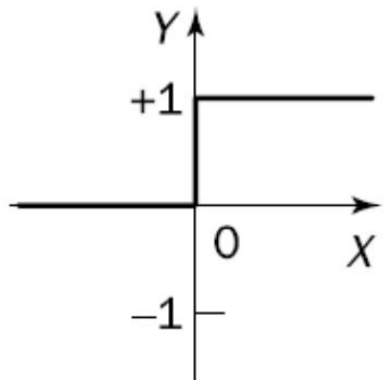
t: ngưỡng kích hoạt

Y: hàm kích hoạt hay hàm truyền – 1 nếu $X \geq t$, 0 nếu ngược lại

Cơ bản về mạng nơ-ron

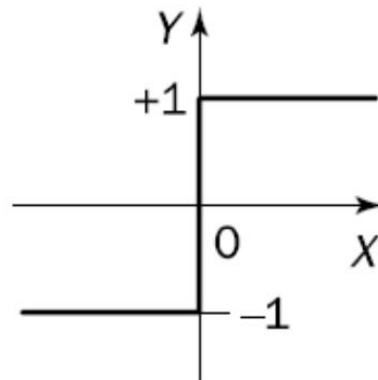
- Hàm lan truyền của Perceptron:
 - Bốn hàm truyền có ứng dụng thực tế

Step function



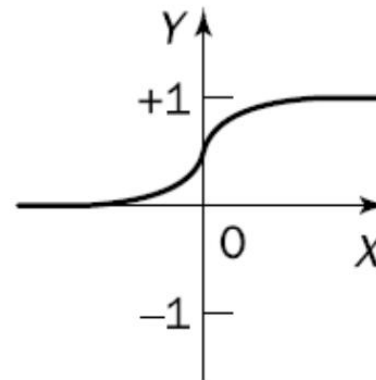
$$Y^{step} = \begin{cases} 1, & \text{if } X \geq 0 \\ 0, & \text{if } X < 0 \end{cases}$$

Sign function



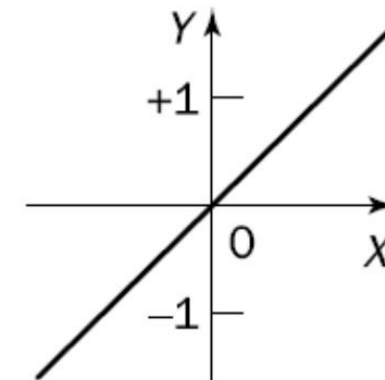
$$Y^{sign} = \begin{cases} +1, & \text{if } X \geq 0 \\ -1, & \text{if } X < 0 \end{cases}$$

Sigmoid function



$$Y^{sigmoid} = \frac{1}{1 + e^{-X}}$$

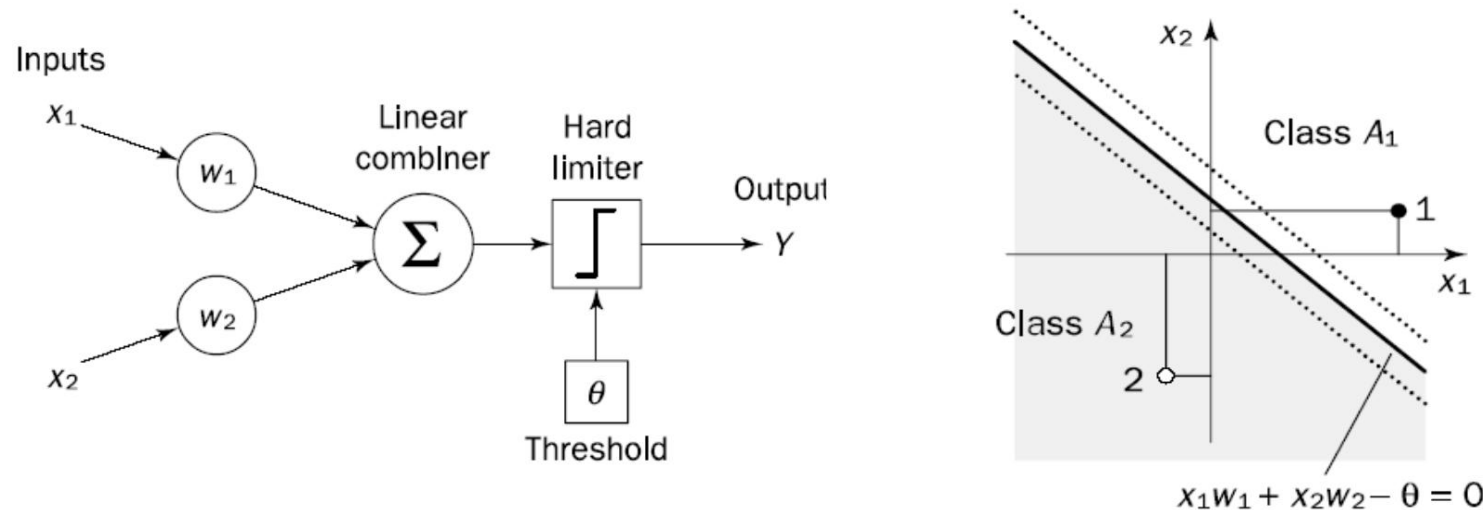
Linear function



$$Y^{linear} = X$$

Perceptron

- Là mạng neuron nhân tạo đơn giản nhất gồm một neuron với các trọng số có thể điều chỉnh được và một giới hạn cứng.



- Perceptron phân loại các input thành hai lớp A_1 và A_2 bị chia cách bởi một siêu phẳng cho bởi hàm phân biệt tuyến tính

$$\sum_{i=1}^n x_i w_i - \theta = 0$$

Perceptron

- Một perceptron học bằng cách điều chỉnh siêu phẳng phân biệt theo sai số $e(p) = Y_d(p) - Y(p)$ với $Y_d/Y(p)$ là kết xuất thực/mong muốn tại bước p .
- Nếu $e(p) > 0 \rightarrow$ tăng $Y(p)$, ngược lại giảm $Y(p)$.
- Nếu $x_i(p) > 0$, tăng $w_i(p)$ sẽ làm tăng $Y(p)$. Ngược lại nếu $x_i(p) < 0$, tăng $w_i(p)$ sẽ làm giảm $Y(p)$.
- Do đó, luật học perceptron có thể được viết là với α là tốc độ học

$$w_i(p + 1) = w_i(p) + \alpha \times x_i(p) \times e(p)$$

Perceptron

- Thuật toán Học:

- Khởi tạo: $(w_1, w_2, \dots, w_n, t) = [-0.5, 0.5]$
- Kích hoạt: tính kết xuất thực sự tại lần lặp $p = 1$

$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) w_i(p) - \theta \right]$$

- Cập nhật trọng số: $w_i(p+1) = w_i(p) + \Delta w_i(p)$
trong đó: $\Delta w_i(p) = \alpha \times x_i(p) \times e(p)$
- Lặp: tăng p lên 1 và lặp lại bước 2.

Perceptron

- Ví dụ:

- Học hàm AND:

x_1	x_2	x_1	x_2	x_1	x_2	x_1	x_2
0	0	0	0	0	0	0	0
0	1	0	0	1	1	1	1
1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0

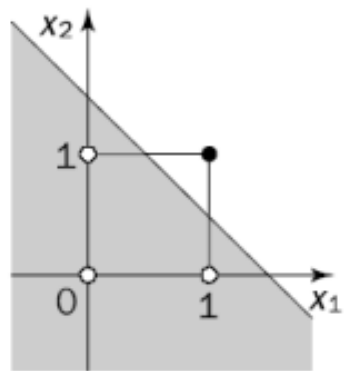
- Khởi tạo trọng số: $w_1 = 0.5$, $w_2 = -0.1$, $\theta = 0.2$, tốc độ học = 0.1

Perceptron

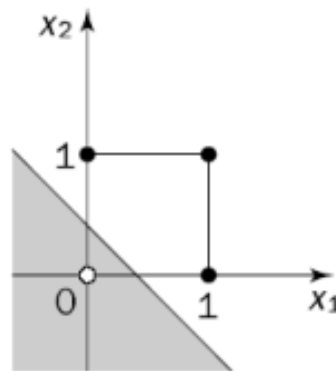
- Nhận xét :
 - Kết xuất của perceptron là một đường thẳng (mặt phẳng) phân biệt tuyến tính:

$$x_1 w_1 + x_2 w_2 = \theta$$

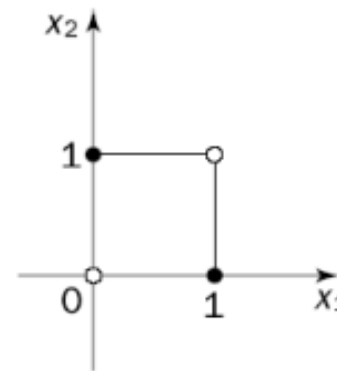
- Do đó, perceptron có thể học được hàm AND, OR nhưng không được hàm XOR



(a) AND ($x_1 \cap x_2$)



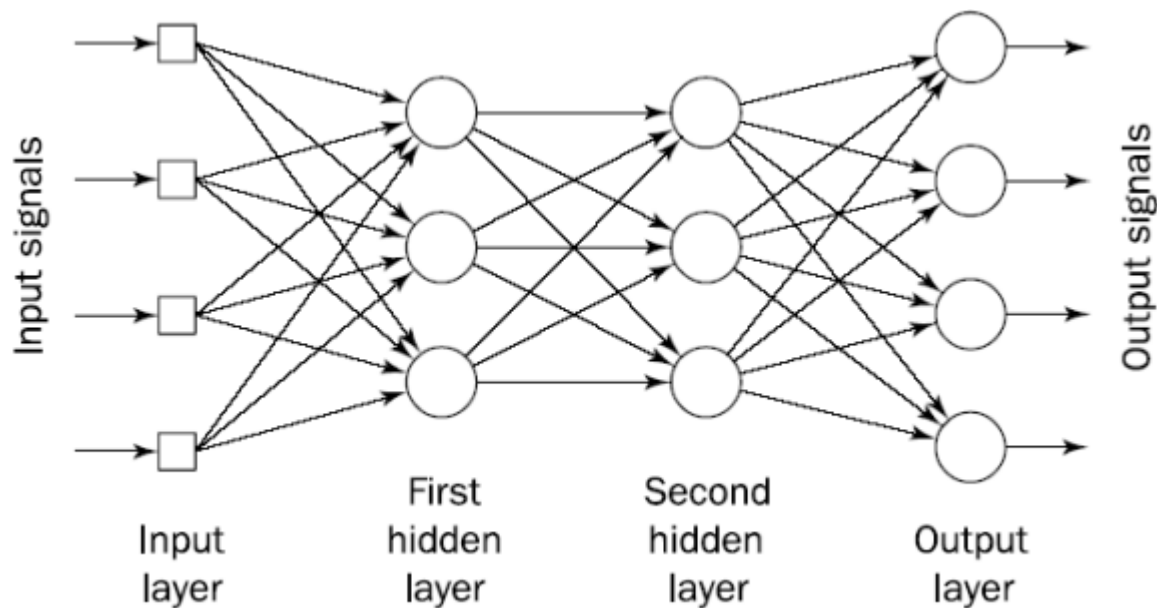
(b) OR ($x_1 \cup x_2$)



(c) Exclusive-OR
($x_1 \oplus x_2$)

Mạng Neuron nhiều lớp

- Một mạng neuron lan truyền tiến gồm một lớp nhập, ít nhất một lớp ẩn, và một lớp xuất.

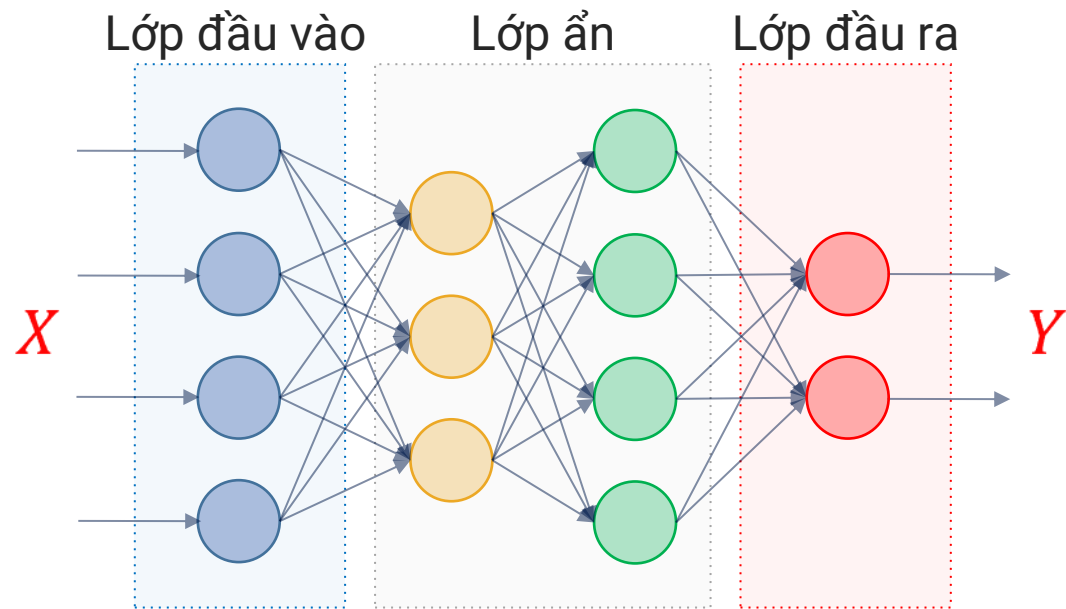


Mạng Neuron nhiều lớp

- Lớp nhập: nhận các input và phân phối chúng cho tất cả neuron trong lớp ẩn
- Lớp xuất: biểu diễn kết quả của toàn mạng
- Lớp ẩn:
 - Dò tìm các đặc trưng.
 - Các neuron trong lớp này “ẩn” các kết xuất mong muốn của chúng.
 - Mạng một lớp ẩn có thể biểu diễn bất kỳ hàm liên tục nào.
 - Mạng hai lớp ẩn có thể biểu diễn các hàm không liên tục.

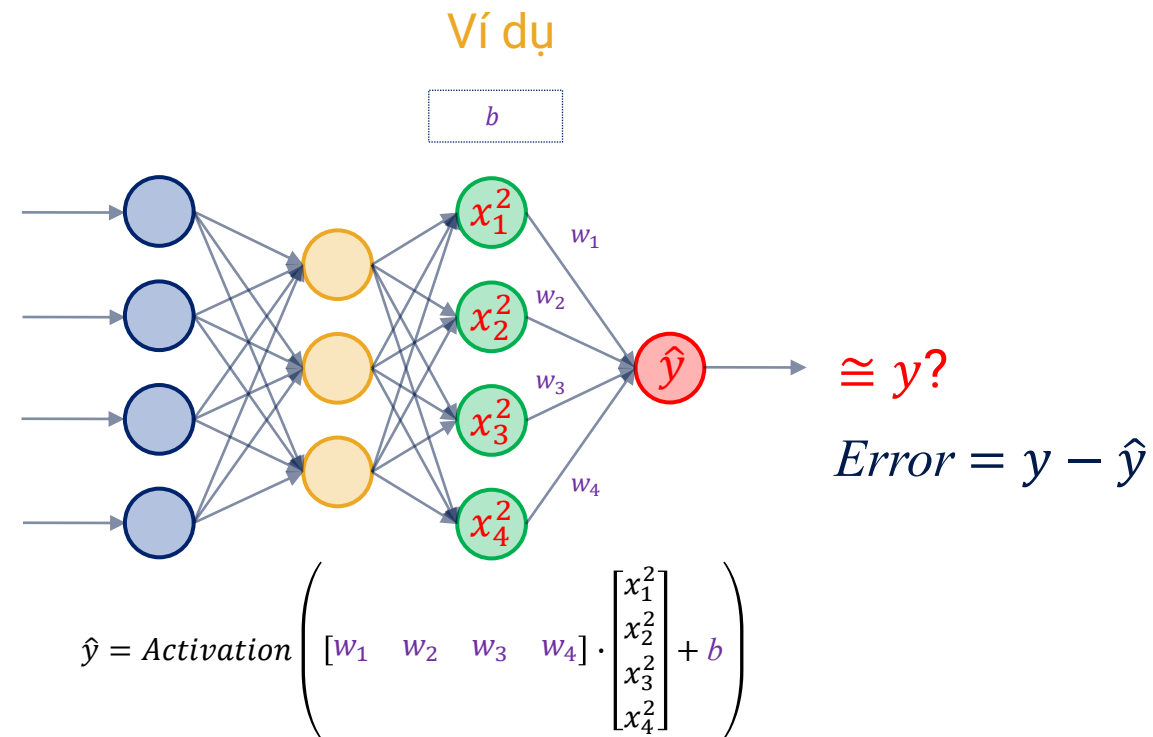
Mạng Neuron nhiều lớp

- Ví dụ:



Mạng Neuron nhiều lớp

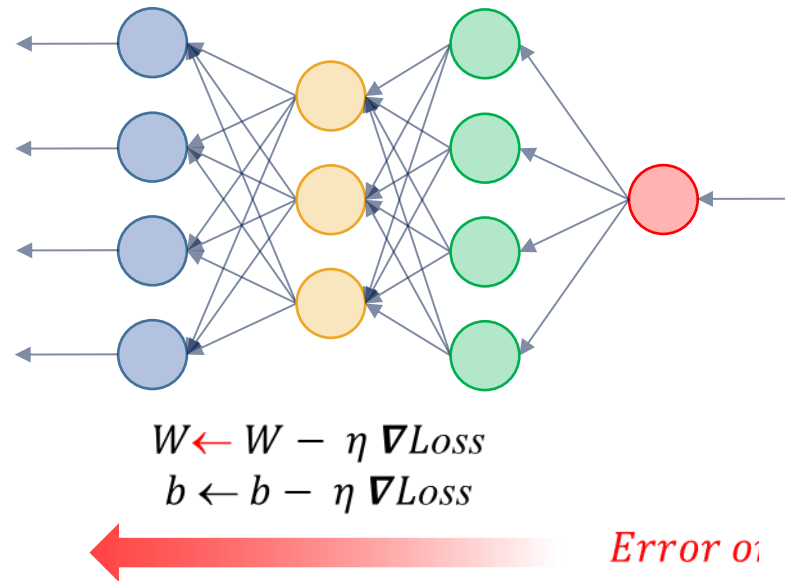
- Học trong mạng neuron nhiều lớp
 - Truyền xuôi



Mạng Neuron nhiều lớp

- Học trong mạng neuron nhiều lớp
 - Truyền ngược

Ví dụ.



Một số vấn đề với học sâu

- **Overfitting**

- Là hiện tượng xảy ra khi một mô hình học quá kỹ dữ liệu huấn luyện đến mức nó không thể tổng quát hóa tốt cho dữ liệu mới.
- Hiểu đơn giản, mô hình trở nên “quá khớp” với dữ liệu huấn luyện và mất khả năng dự đoán chính xác trên dữ liệu chưa từng thấy
- Dấu hiệu nhận biết:
 - Mô hình đạt độ chính xác rất cao trên tập huấn luyện nhưng lại có hiệu suất kém trên tập kiểm thử.
 - Độ sai số thấp trên dữ liệu huấn luyện nhưng cao trên dữ liệu kiểm thử

Một số vấn đề với học sâu

- **Overfitting**

- Nguyên nhân:

- Mô hình quá phức tạp: Sử dụng mạng nơ-ron có quá nhiều tầng hoặc tham số cho một bài toán đơn giản.
- Dữ liệu huấn luyện không đủ: Khi dữ liệu quá ít, mô hình dễ học thuộc thay vì học khái quát.
- Dữ liệu nhiều: Mô hình học cả những điểm bất thường không đại diện cho xu hướng chung

Một số vấn đề với học sâu

- **Overfitting**

- Khắc phục:

- Regularization: Áp dụng kỹ thuật như L1/L2 regularization để giảm độ phức tạp của mô hình.
- Dropout: Loại bỏ ngẫu nhiên một số neuron trong quá trình huấn luyện để tránh phụ thuộc quá mức.
- Cross-validation: Kiểm tra mô hình trên nhiều tập dữ liệu khác nhau để đánh giá khả năng tổng quát.
- Early stopping: Dừng huấn luyện sớm khi hiệu suất trên tập kiểm thử bắt đầu giảm.

Một số vấn đề với học sâu

• Underfitting

- Là hiện tượng xảy ra khi mô hình học máy hoặc học sâu quá đơn giản để nắm bắt được các đặc điểm quan trọng trong dữ liệu.
- Kết quả là mô hình không học được mối quan hệ giữa đầu vào và đầu ra, dẫn đến hiệu suất kém trên cả tập huấn luyện và kiểm thử
- Nhận biết:
 - Độ chính xác thấp trên cả tập huấn luyện và kiểm thử.
 - Sai số cao và không cải thiện nhiều qua các epoch huấn luyện.
 - Mô hình không thể nắm bắt xu hướng hoặc quy luật trong dữ liệu.

Một số vấn đề với học sâu

- **Underfitting**

- Nguyên nhân:

- Mô hình quá đơn giản

Ví dụ: như dùng hồi quy tuyến tính cho dữ liệu có quan hệ phi tuyến.

- Huấn luyện chưa đủ lâu: Mô hình chưa có đủ thời gian để học.
 - Regularization quá mạnh: Làm giảm khả năng học của mô hình.
 - Lựa chọn đặc trưng kém: Không cung cấp đủ thông tin cần thiết cho mô hình

Một số vấn đề với học sâu

- **Underfitting**

- Cách khắc phục underfitting

- Tăng độ phức tạp của mô hình: Thêm tầng, node hoặc sử dụng kiến trúc mạnh hơn.
 - Huấn luyện lâu hơn: Cho mô hình thêm thời gian để học.
 - Giảm regularization: Cho phép mô hình học nhiều hơn.
 - Thêm đặc trưng đầu vào: Cung cấp thêm thông tin hữu ích cho mô hình.



Deep Learning với TensorFlow

Giới thiệu về TensorFlow

- TensorFlow là một thư viện mã nguồn mở mạnh mẽ được phát triển bởi Google
- Chuyên dùng cho các ứng dụng machine learning và deep learning.
- Giúp các nhà phát triển và nhà nghiên cứu xây dựng, huấn luyện và triển khai các mô hình học máy một cách hiệu quả và linh hoạt.



Giới thiệu về TensorFlow

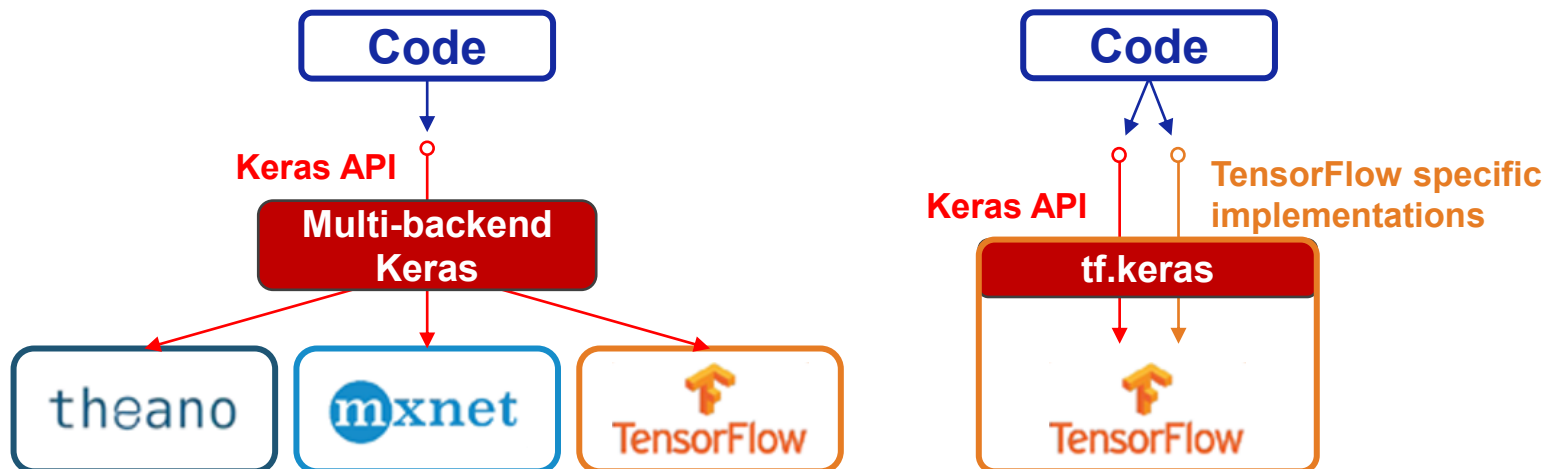
- Ra đời: TensorFlow được phát hành lần đầu vào năm 2015 bởi nhóm Google Brain.
- Ngôn ngữ: Viết bằng C++ nhưng cung cấp giao diện lập trình bằng Python, rất thân thiện với người dùng.
- Ứng dụng: Từ nhận diện hình ảnh, xử lý ngôn ngữ tự nhiên, dự đoán chuỗi thời gian đến các hệ thống đề xuất.
- Cài đặt TensorFlow

```
> pip install tensorflow
```

```
> pip install tensorflow==[desired-version]
```

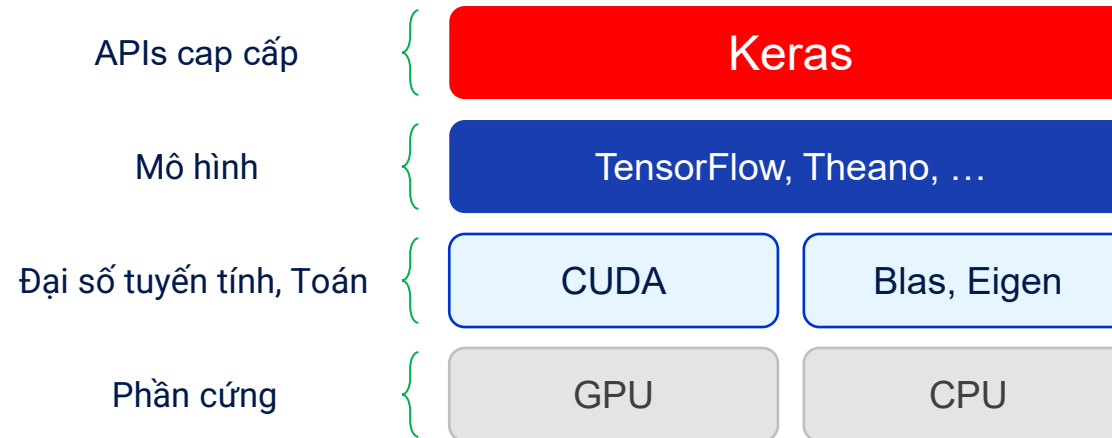
Giới thiệu về Keras

- Keras là một thư viện mã nguồn mở cấp cao trong lĩnh vực học sâu (deep learning), được viết bằng Python.
- Cung cấp API cấp cao cho phép người dùng xây dựng, huấn luyện, đánh giá và thực thi tất cả các loại mạng nơ-ron trên Backend TensorFlow



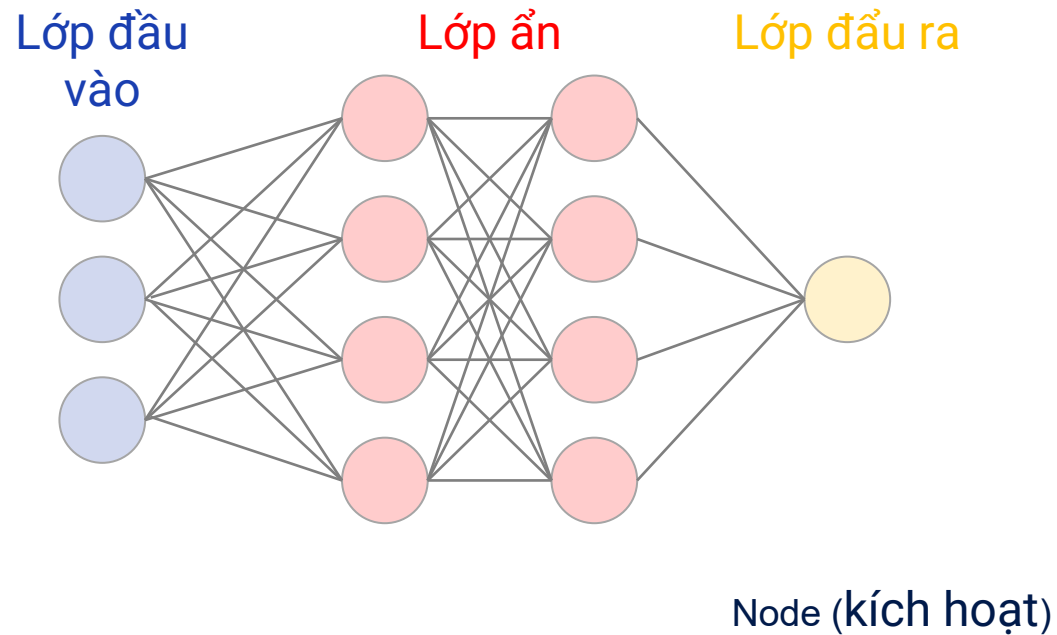
Giới thiệu về Keras

- Keras



Giới thiệu về Keras

- Dễ dàng để mô hình hóa mạng nơ-ron sâu với Keras.



Giới thiệu về Keras

- Quy trình đào tạo

Xác định cấu trúc lớp của mô hình (chỉ định các chức năng kích hoạt).



Xác định hàm mất mát và phương pháp tối ưu hóa.



Dịch mô hình.



Chỉ định phương pháp học hàng loạt và bắt huấn luyện (giảm dần độ dốc).

Giới thiệu về Keras

- Hàm mất mát trong Keras

Tên	Công thức	Biểu thức Keras
Squared Error (L2 Error)	$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$	loss = "mse"
Absolute Error (L1 Error)	$L = \sum_{i=1}^n y_i - \hat{y}_i $	loss = "mae"
Binary Cross Entropy	$L = -\frac{1}{n} \sum_{i=1}^n (y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i))$	loss = "binary_crossentropy"
Multi-Class Cross Entropy	$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik})$	loss = "categorical_crossentropy"

Giới thiệu về Keras

- Thuật toán giảm độ dốc

Thuật toán	Giải thích	Keras
Stochastic Gradient Descent	Thuật toán giảm độ dốc với việc học theo lô	SGD(lr=0.1)
Momentum	Có tính đến độ dốc từ bước trước	SGD(momentum=0.9)
Nesterov Momentum	Giảm thiểu những chuyển động không cần thiết	SGD(lr=0.01, momentum=0.9, nesterov=True)
Adagrad	Kích thước bước thích ứng	Adagrad(lr=0.01)
RMSProp	Cải thiện trên Adagrad	RMSProp(lr=0.01)
Adam	Kết hợp các thuộc tính tốt nhất của Adagrad và RMSProp	Adam(lr=0.01)

Giới thiệu về Keras

- Hàm kích hoạt

Tên	Công thức	Keras
Linear	$linear(x) = x$	activation="linear"
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	activation="sigmoid"
Tanh	$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	activation="tanh"
ReLu	$ReLu(x) = \max(0, x)$	activation="relu"
Softmax	$\sigma(x)_j = \frac{e^{x_j}}{\sum_{k=1}^K e^{x_k}}$	activation="softmax"

Giới thiệu về Keras

- Các Layer

Keras	Giải thích
Dense()	Dense layer
Conv1D()	1D convolution layer
Conv2D()	2D convolution layer
MaxPooling1D()	1D max pooling layer
MaxPooling2D()	2D max pooling layer
Dropout()	Applies dropout to the input
Flatten()	Flattens the input
Embedding()	Turns positive integers (label encoding) into dense vectors
SimpleRNN()	RNN where the output is fed back to input
LSTM()	LSTM layer
TimeDistributed()	A wrapper that applies a layer to every time slice
Input()	The first layer for a functional API model

Xây dựng mô hình với Keras

- API tuần tự:
 - Các lớp được thêm vào đối tượng “Tuần tự”.
 - Dễ dàng nhưng kém linh hoạt.
- API chức năng:
 - Lớp đầu tiên phải được xác định là “Đầu vào”, nơi hình dạng dữ liệu được chỉ định.
 - Mỗi lớp được định nghĩa là một hàm lấy làm đối số của lớp trước đó.
 - Đối tượng “Model” nhận làm đối số ở lớp đầu tiên và lớp cuối cùng.
 - Linh hoạt hơn.

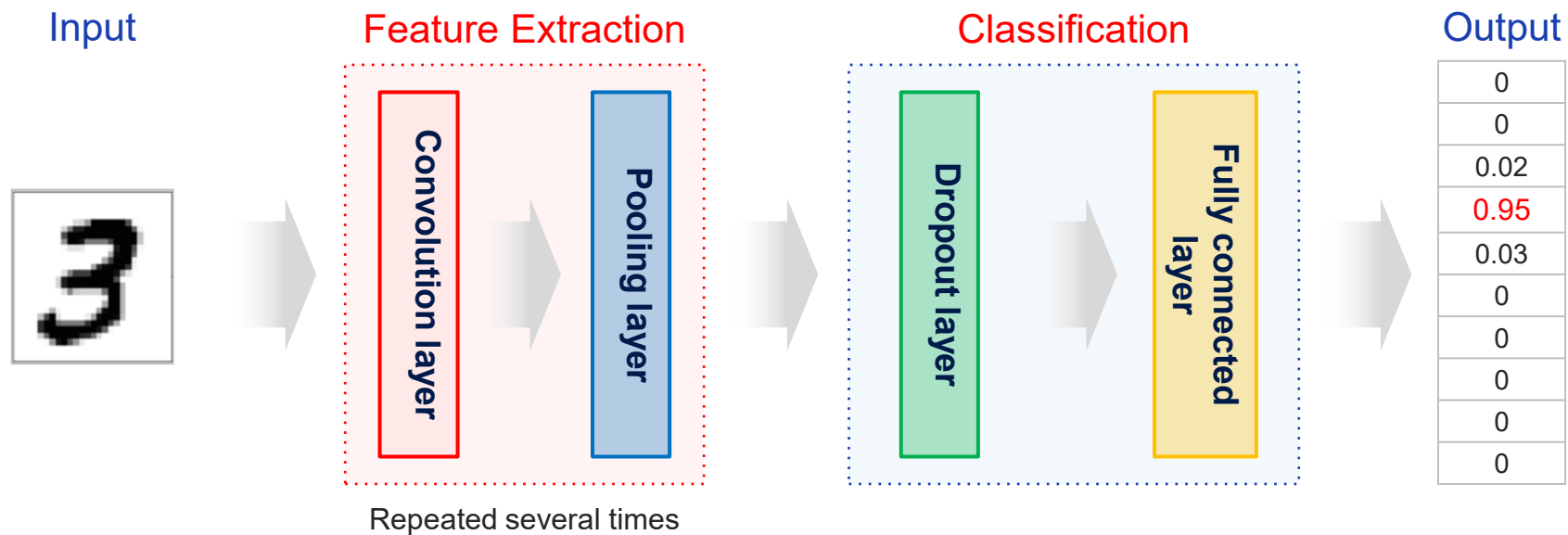
Xây dựng mô hình với Keras

- Bộ dữ liệu được cung cấp bởi Keras

Tên	Giải thích
cifar10	Hình ảnh màu 32×32 được phân loại thành 10 danh mục. 50000 hình ảnh đào tạo và 10000 hình ảnh thử nghiệm.
cifar100	Hình ảnh màu 32×32 được phân loại theo 100 danh mục. 50000 hình ảnh đào tạo và 10000 hình ảnh thử nghiệm.
imdb	25000 bài đánh giá phim từ IMDB có gắn nhãn nhị phân (tích cực hoặc tiêu cực).
reuters	11228 bản tin từ Reuters, được dán nhãn trên 46 chủ đề.
mnist	Hình ảnh thang độ xám 28×28 của các chữ số viết tay. 60000 hình ảnh đào tạo và 10000 hình ảnh thử nghiệm.
fashion_mnist	Hình ảnh thang độ xám 28×28 của 10 danh mục thời trang. 60000 hình ảnh đào tạo và 10000 hình ảnh thử nghiệm.
boston_housing	13 thuộc tính của những ngôi nhà tại các địa điểm khác nhau quanh vùng ngoại ô Boston vào cuối những năm 1970.

CNN với Keras

- Các lớp chập và gộp được lặp lại nhiều lần.
- Có một lớp được kết nối đầy đủ ngay trước đầu ra.



CNN với Keras

- Đọc dữ liệu

```
In [132]: 1 import numpy as np
          2 import matplotlib.pyplot as plt
          3 import warnings
          4 from sklearn.model_selection import train_test_split
          5 from tensorflow.keras.datasets.mnist import load_data
          6 from tensorflow.keras.models import Sequential
          7 from tensorflow.keras.layers import Dense, Conv2D, Dropout, Flatten, MaxPooling2D
          8
          9 from tensorflow.keras.optimizers import Adam, RMSprop, SGD
         10 warnings.filterwarnings('ignore')
         11 %matplotlib inline
```

CNN với Keras

- Đọc dữ liệu

- Đưa dữ liệu vào.

```
In [133]: (X_train, y_train), (X_test, y_test) = load_data()  
          n_train_size = X_train.shape[0]
```

- Xem các hình dạng.

```
In [134]: print("-"*50)  
          print("Training data X shape: {}".format(X_train.shape))  
          print("Training data y shape: {}".format(y_train.shape))  
          print("-"*50)  
          print("Test data X shape: {}".format(X_test.shape))  
          print("Test data y shape: {}".format(y_test.shape))  
          print("-"*50)
```

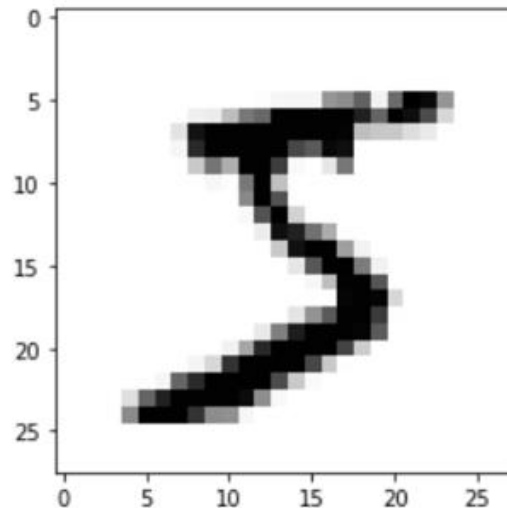
```
-----  
Training data X shape: (60000, 28, 28)  
Training data y shape: (60000,)  
-----
```

```
Test data X shape: (10000, 28, 28)  
Test data y shape: (10000,)  
-----
```

CNN với Keras

- Trực quan dữ liệu

```
In [135]: 1 i_image = 0  
          2 plt.imshow(X_train[i_image,:,:],cmap="Greys")  
          3 plt.show()
```



Line 135-1

- You may change this at will.

CNN với Keras

- Chuẩn bị dữ liệu

- Scaling

```
In [136]: X_train = X_train/255  
X_test = X_test/255
```

- Định hình lại

```
In [137]: X_train = X_train.reshape(-1,28,28,1)  
X_test = X_test.reshape(-1,28,28,1)
```

CNN với Keras

- Chuẩn bị dữ liệu

- Mã hóa one-hot

```
In [139]: y = np.concatenate([y_train,y_test],axis=0)
          y = tf.keras.utils.to_categorical(y,10)
          y_train = y[:n_train_size,:]
          y_test = y[n_train_size:,:]
```

CNN với Keras

- Xây dựng mô hình

```
In [140]: 1 drop_prob = 0.5
          2 my_model = Sequential()
          3 # 1st convolution + pooling.
          4 my_model.add(Conv2D(input_shape=(28,28,1), filters=32, kernel_size=(5,5), padding='same', activation="relu"))
          5 my_model.add(MaxPooling2D(pool_size=2))
          6 # 2nd convolution + pooling.
          7 my_model.add(Conv2D(filters=64, kernel_size=(5,5), padding='same', activation="relu"))
          8 my_model.add(MaxPooling2D(pool_size=2))
          9 # Flattened fully connected layer.
         10 my_model.add(Flatten())
         11 my_model.add(Dense(units = 1024, activation="relu"))
         12 # Apply dropout.
         13 my_model.add(Dropout(rate=drop_prob))
         14 # Output layer.
         15 my_model.add(Dense(units = 10, activation="softmax"))
```



Line 140-1

- 1 channel of grayscale

CNN với Keras

- Xây dựng mô hình

```
In [141]: my_model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	832
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	51264
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
flatten (Flatten)	(None, 3136)	0
dense_3 (Dense)	(None, 1024)	3212288
dropout (Dropout)	(None, 1024)	0
dense_4 (Dense)	(None, 10)	10250

Total params: 3,274,634

Trainable params: 3,274,634

Non-trainable params: 0

CNN với Keras

- Xác định các siêu tham số và trình tối ưu hóa

```
In [142]: n_epochs = 10  
          batch_size = 200  
          learn_rate = 0.001
```

```
In [143]: my_optimizer=Adam(lr=learn_rate)  
          my_model.compile(loss = "categorical_crossentropy", optimizer = my_optimizer, metrics=["accuracy"])
```

CNN với Keras

- Huấn luyện

```
In [144]: my_summary = my_model.fit(X_train, y_train, epochs=n_epochs, batch_size = batch_size, validation_split = 0.2, verbose = 1)
```

Epoch 1/10
240/240 [=====] - 42s 162ms/step - loss: 0.4622 - accuracy: 0.8538 - val_loss: 0.0525 - val_accuracy: 0.9837
Epoch 2/10
240/240 [=====] - 39s 162ms/step - loss: 0.0607 - accuracy: 0.9819 - val_loss: 0.0533 - val_accuracy: 0.9835
Epoch 3/10
240/240 [=====] - 39s 164ms/step - loss: 0.0419 - accuracy: 0.9873 - val_loss: 0.0360 - val_accuracy: 0.9893
Epoch 4/10
240/240 [=====] - 39s 161ms/step - loss: 0.0296 - accuracy: 0.9908 - val_loss: 0.0441 - val_accuracy: 0.9872
Epoch 5/10
240/240 [=====] - 39s 160ms/step - loss: 0.0233 - accuracy: 0.9923 - val_loss: 0.0349 - val_accuracy:



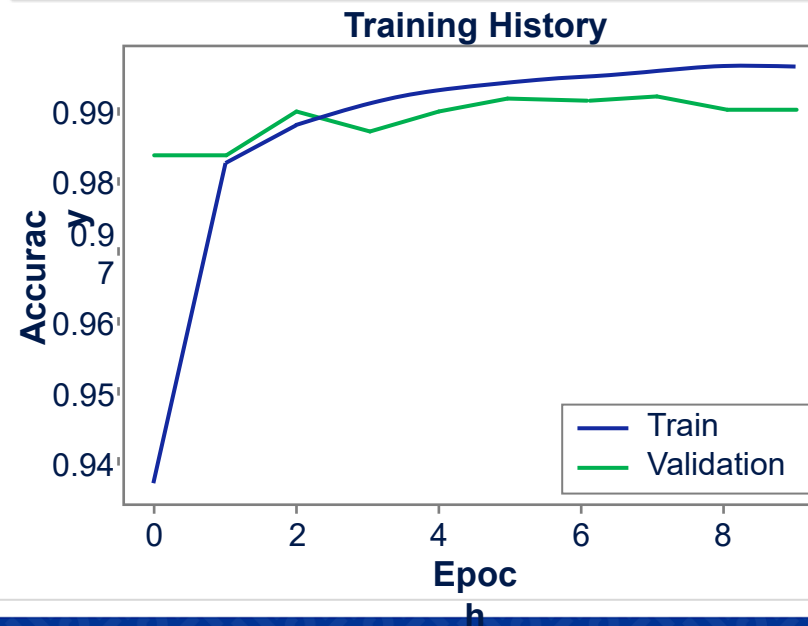
Line 144

- verbose = 0 means no output. verbose = 1 to view the epochs.

CNN với Keras

- Trực quan hóa quá trình huấn luyện

```
In [145]: n_skip = 0  
plt.plot(my_summary.history['accuracy'][n_skip:], c="b")  
plt.plot(my_summary.history['val_accuracy'][n_skip:], c="g")  
plt.title('Training History')  
plt.ylabel('Accuracy')  
plt.xlabel('Epoch')  
plt.legend(['Train', 'Validation'], loc='lower right')  
plt.show()
```



CNN với Keras

- Kiểm thử

```
In [146]: ACC = my_model.evaluate(X_test, y_test, verbose=0)[1]
          print("Test Accuracy : {}".format(np.round(ACC,3)))
          Test Accuracy : 0.992
```

Bài tập thực hành

HỎI & ĐÁP

